

Design Note — LLD Practice Platform

Design Note — LLD Practice Platform

1. MVP scope

A modular monolith that supports one end-to-end practice loop:

1. Browse four LLD problems.
2. Start an attempt and edit a structured design (assumptions, classes, relationships, tradeoffs, extension).
3. Save draft / submit.
4. Receive rubric findings (async).
5. Review history and retry.

Stack: Express + TypeScript + Prisma/SQLite + React/Vite + OpenAI (with HeuristicEvaluator fallback).

2. User flow

Problems !' Studio (draft) !' Submit (202) !' Attempt detail (poll) !' Feedback
!~ History !'-----!™

Identity for the prototype is a httpOnly learnerId cookie (UUID). No login.

3. Important types and responsibilities

Type Responsibility

--- ---

Problem Statement, constraints, extension prompt, attached rubric JSON

Attempt One practice pass; owns status lifecycle

Submission Immutable snapshot of DesignArtifact + format text_v1

DesignArtifact Structured learner fields

Rubric / criteria Stable scoring dimensions

Evaluation / Finding Per-criterion score, evidence, concern, suggestion, confidence

Evaluator (port) evaluate(input) !' EvaluationResult

HeuristicEvaluator Deterministic-ish scoring without an API key

OpenAiEvaluator LLM JSON rubric scoring

EvaluationQueue Serial in-process async jobs

PracticeService Start, draft, submit, retry, history, run evaluation

Domain validation and state transitions live in attemptLifecycle (no Express/OpenAI imports).

State machine

draft !' submitted !' evaluating !' completed failed

- Failed keeps the submission; retry-evaluation re-queues.
- Double submit !' conflict (409).

4. Evaluation approach

Deterministic (always): required sections present, minimum length, illegal transitions, idempotent enqueue, persist submission + pending evaluation before the worker runs.

Judgment (LLM when OPENAI_API_KEY is set): fixed rubric in the prompt; JSON object output; evidence must reference learner text; low temperature.

Fallback: HeuristicEvaluator when no key / EVALUATOR=heuristic. UI shows evaluator name so feedback provenance is honest.

5. Change tests

A — New submission format (e.g. diagram): add a DesignArtifact / payload variant and a format-aware evaluator path. Attempt.submit still persists a Submission; practice HTTP flow unchanged.

B — New evaluator (rules / human): implement Evaluator and select it in createEvaluator. PracticeService depends only on the port.

6. API surface

- GET /api/problems, GET /api/problems/:slug
- POST /api/attempts, GET PATCH /api/attempts/:id
- POST /api/attempts/:id/submit !' 202
- POST /api/attempts/:id/retry-evaluation !' 202
- GET /api/history
- GET /api/health

7. Trade-offs and scale judgement

- Monolith over microservices: assignment is an LLD/domain exercise; one process keeps the practice loop obvious.
- SQLite over hosted DB: zero ops for a prototype; submissions survive evaluator failure on disk.
- In-process queue over Redis/Bull: enough for demo concurrency; jobs are not durable across process restart (documented limitation).
- Structured text over diagrams/code: maximizes signal per engineering hour.

If AI evaluation becomes slow under load, extract the evaluator worker first (same DB, same Evaluator interface) — not a fleet of microservices.

8. Security (prototype)

Helmet, JSON body size limit (256kb), server-only API key, httpOnly learner cookie (not a real auth boundary).